Document number	P3052R0
Date	2023-11-16
Audience	LEWG, SG23 (Safety and Security), SG9 (Ranges)
Reply-to	Hewill Kang <hewillk@gmail.com>

`view_interface::at()`

Abstract

This paper provides the `at()` method to `ranges::view_interface` to provide a safe access method for the view class.

Revision history

R0

Initial revision.

Discussion

Currently, the committee adopted P2821 in C++26, which adds a missing `at()` to `std::span` to consistent its API with other containers as well as `std::string_view`. Given that the two standard views `span` and `string_view` now have the `at()` method, The author thinks it's time to extend this further to generic views in `<ranges>`, which will bring:

1. Consistency. P1739 brings interaction between range adaptors and `span`/`string_view`, suggesting that it makes sense to maintain API consistency. Users do not need to worry about missing functionality when converting from the first two to view such as `subrange`.

Table — Standard range types and access APIs

Element access	`operator[]`	`at`	`front`	`back`	`data`
`string/array/vector`	✅	✅	✅	✅	✅
`string_view/span`	✅	✅	✅	✅	✅
`ranges::`*`meow`*`_view`	✅	❌	✅	✅	✅

2. Safety. Compared with standard containers, view classes do not provide any bounds checking, which makes indexing access unsafe and discourages third-party projects that focus on security. The `at()` method can be a turning point.

Design

How to make all view classes provide `at()` (if they can)?

All range factories/adaptors in `<ranges>` (including `std::generator`) are derived from `view_interface`, this is intended to synthesize more members through `view_interface` when they model a specific range concept.

For example, a derived class that satisfies `forward_range` will have an available `front()` even if the implementation does not provide one. This makes it intuitive for users to spell something like `views::single(0).front()` to get the first element.

In addition, thanks to LWG 3549 reducing the size of range adaptors caused by unnecessary padding, views only need to inherit `view_interface` instead of `view_base` to be enabled. The author believes that any generic views should prefer to inherit from `view_interface`, such a feature of automatically inheriting functionality is very valuable.

This is what P2278 does by introducing `cbegin()`/`cend()` for views. Even though the derived class may not currently gain any benefit from `view_interface`, this does not mean that `view_interface` will not add new members or relax the constraints of some members, just as LWG 3715 makes `input_ranges` also have `empty()`.

To sum up, the author believes that the implementation of `at()` can just by adding constrained members to `view_interface`.

When is `at()` provided?

Since `at()` is a random access operation, the view type needs to model `random_access_range`; we also need to know the size of the range for boundary checking, which requires `sized_range`.

What is the parameter type of the function?

There are three possible candidates for the parameter type of `at()`.

The first is `range_size_t<R>`, which is the return type of `ranges::size` used to query range boundaries. However, since the signedness of this type is unspecified, and it is not closely related to the iterator's difference type which is involved in the implementation, the author does not consider it to be a suitable option.

So the question becomes, should the parameter type be signed i.e. `range_difference_t<R>`, or unsigned i.e. *`make-unsigned-like-t`*`<range_difference_t<R>>`? The author believes that the former is a better choice, as it maintains a consistent interface with the `operator[]` and eliminates the need for the additional signedness conversion.

What are the conditions for boundary checking?

When the index value `n < 0` or `n >= ranges::distance(r)`, it can be considered out of bounds.

Although the underlying iterator-based formula implies that it works with negative signed integers, e.g. `subrange(v.begin() + 1, v.end())[-1]` legally points to the first element, the author believes that this kind of access cannot generally be regarded as a safe operation because `v.begin()` is indeed excluded from the originally intended scope, in which case throwing an exception is more likely to catch user errors.

Since `ranges::distance(r)` is specified to return the signed value of `ranges::size(r)` when `r` is a `sized_range`, based on the consideration of reducing unnecessary type conversions in the previous discussion, the author prefers to use `ranges::distance(r)` in the condition.

Proposed change

This wording is relative to N4964.

1. Add a new feature-test macro to 17.3.2 [version.syn]:

```
#define __cpp_lib_view_interface_at_2023XXL // also in <ranges>
```

2. Modify 26.5.3 [view.interface] as indicated:

```
namespace std::ranges {
  template<class D>
    requires is_class_v<D> && same_as<D, remove_cv_t<D>>
  class view_interface {
    […]
  public:
    […]

    template<random_access_range R = D>
      constexpr decltype(auto) operator[](range_difference_t<R> n) {
        return ranges::begin(derived())[n];
      }
    template<random_access_range R = const D>
      constexpr decltype(auto) operator[](range_difference_t<R> n) const {
        return ranges::begin(derived())[n];
      }

    template<random_access_range R = D> requires sized_range<R>
      constexpr decltype(auto) at(range_difference_t<R> n);
    template<random_access_range R = const D> requires sized_range<R>
      constexpr decltype(auto) at(range_difference_t<R> n) const;
  };
}

[…]

  template<random_access_range R = D> requires sized_range<R>
    constexpr decltype(auto) at(range_difference_t<R> n);
  template<random_access_range R = const D> requires sized_range<R>
    constexpr decltype(auto) at(range_difference_t<R> n) const;

-?- Returns: operator[](n).

-?- Throws: out_of_range if n < 0 or n >= ranges::distance(derived()).
```

References

[P2821R4]
Jarrad J. Waterloo. `span.at()`. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2821r4.html

[P1739R4]
Hannes Hauswedell. Avoid template bloat for `safe_ranges` in combination with ‘subrange-y’ view adaptors. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1739r4.html

[LWG 3549]
Tim Song. LWG 3549: `view_interface` is overspecified to derive from `view_base`. URL: https://cplusplus.github.io/LWG/issue3549

[P2278R4]

Barry Revzin. `cbegin` should always return a constant iterator. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2278r4.html

[LWG 3715]
Hewill Kang. LWG 3715: `view_interface::empty` is overconstrained. URL: https://cplusplus.github.io/LWG/issue3715

Acknowledgements

Thanks to Arthur O'Dwyer for sharing his valuable perspective on the maillist.